

Universidade Estadual de Campinas

Instituto de Computação

MC714 - Sistemas Distribuídos

Relatório técnico do Trabalho 2

Implementação de relógios lógicos, exclusão mútua e eleição de líder com HTTP/JSON entre contêineres

Autores: José Eduardo Santos Rabelo (RA 260551)
Raphael Salles Vitor de Souza (RA 223641)
Professor: Luiz Fernando Bittencourt
Disciplina: MC714 - Sistemas Distribuídos
Semestre: 1º semestre de 2026
Repositório: RaphaelSVSouza/MC714-Trabalho-2

Resumo

Este relatório descreve a implementação de um sistema distribuído didático composto por três nós independentes e um observador externo, todos executados em contêineres e comunicando-se por HTTP/JSON. O projeto reúne três mecanismos clássicos: relógios lógicos de Lamport, exclusão mútua distribuída baseada em Ricart–Agrawala e eleição de líder baseada no algoritmo Bully. O texto apresenta a arquitetura, o modelo de mensagens e estado, as adaptações necessárias ao transporte HTTP, as invariantes implementadas, os testes unitários, os smoke tests e os experimentos versionados. Também distingue eventos lógicos de anotações de observabilidade e registra as limitações do modelo, como participantes fixos, ausência de tolerância a partições e dependência de timeouts compatíveis com a rede local.

Palavras-chave: sistemas distribuídos; relógio lógico; exclusão mútua; eleição de líder; HTTP; Docker.

Sumário

1	Introdução	1
2	Objetivos, requisitos e fundamentação	2
2.1	Objetivo geral	2
2.2	Requisitos funcionais	2
2.3	Requisitos de qualidade	2
2.4	Relógios lógicos de Lamport	3
2.5	Exclusão mútua de Ricart–Agrawala	3
2.6	Eleição pelo algoritmo Bully	4
2.7	Escopo e exclusões	4
3	Arquitetura do sistema	4
3.1	Visão geral	4
3.2	Componentes e responsabilidades	5
3.3	Estrutura interna de um nó	6
3.4	Envelope de mensagens	6
3.5	API e observabilidade	7
3.6	Configuração temporal	8
4	Desenvolvimento e implementação	8
4.1	Base de execução	8
4.2	Estado local e sincronização entre corrotinas	8
4.3	Relógio de Lamport e log de eventos	9

4.3.1	Operações do relógio	9
4.3.2	Eventos e anotações	9
4.3.3	Exemplo causal	10
4.4	Exclusão mútua distribuída	10
4.4.1	Modelagem de estados	10
4.4.2	Criação e disseminação do pedido	11
4.4.3	Regra de prioridade	11
4.4.4	Entrada, saída e abort	11
4.4.5	Observador externo	11
4.5	Eleição de líder	12
4.5.1	Mensagens e fluxo	12
4.5.2	Detecção de falha e recuperação	12
4.5.3	Aceitação de líder	12
4.5.4	Limites do identificador de eleição	13
4.6	Decisões de implementação e limitações	13
5	Validação, testes e experimentos	13
5.1	Estratégia de validação	13
5.2	Testes unitários	14
5.3	Smoke tests	14
5.4	Metodologia experimental	15
5.5	Resultados experimentais versionados	16
5.6	Análise dos resultados	16
5.6.1	Lamport	16
5.6.2	Exclusão mútua	16
5.6.3	Eleição	16
5.7	Reprodutibilidade	17
5.8	Ameaças à validade	17
6	Conclusão	18
6.1	Limitações e possíveis extensões	18
7	Fontes de código e declaração de autoria	19
8	Uso de ferramentas de inteligência artificial	20
9	Referências	21

Lista de Figuras

1	Topologia lógica e serviços expostos no host. As arestas bidirecionais entre os nós formam uma malha completa e transportam mensagens de aplicação, mutex, eleição e heartbeat.	5
2	Organização interna de cada nó em camadas. A API delega o transporte HTTP a <code>transport.py</code> e concentra o estado em <code>NodeState</code> , que aplica as funções puras de <code>clock.py</code> , <code>mutex.py</code> e <code>election.py</code>	6

Lista de Tabelas

1	Divisão de responsabilidades no código.	5
2	Envelope comum às mensagens entre nós.	7
3	Endpoints principais dos nós.	7
4	Timeouts configurados no Docker Compose.	8
5	Estados locais usados para Ricart–Agrawala.	10
6	Mensagens do subsistema de eleição.	12
7	Cenários cobertos pelos smoke tests.	15
8	Resumo da execução experimental versionada.	16
9	Fontes de código de terceiros e alterações realizadas.	19

1 Introdução

Sistemas distribuídos são formados por processos independentes que cooperam por troca de mensagens e que, em geral, não compartilham um relógio físico global nem uma memória comum. Essa ausência de uma visão instantânea e única do sistema torna necessárias abstrações para representar causalidade, coordenar o acesso a recursos e escolher um processo responsável por determinadas funções.

O Trabalho 2 da disciplina MC714 materializa esses problemas em uma implementação local, executável e observável. O sistema possui três nós homogêneos — **node1**, **node2** e **node3** — e um serviço auxiliar chamado **resource**. Cada nó é um processo independente, mantém somente estado local em memória e troca mensagens reais por HTTP/JSON. A pilha é orquestrada com Docker Compose, permitindo parar e recuperar processos sem substituir a comunicação distribuída por objetos no mesmo processo ou arquivos compartilhados.

A implementação integra três mecanismos clássicos. O primeiro é o relógio lógico de Lamport, usado para associar timestamps a eventos e preservar a condição de precedência causal: quando um evento pode ter influenciado outro, seu timestamp deve ser menor [1]. O segundo é a exclusão mútua distribuída de Ricart–Agrawala, na qual cada processo solicita autorização aos demais e desempata pedidos concorrentes por uma ordem total [2]. O terceiro é uma adaptação do algoritmo Bully, que elege como coordenador o processo ativo de maior identificador [3].

Além de implementar os algoritmos, o projeto busca tornar o comportamento auditável. Para isso, expõe endpoints de inspeção, registra um histórico local de eventos, possui testes unitários, executa smoke tests sobre a pilha real e gera resultados estruturados em JSON e CSV. O modelo também distingue explicitamente evento lógico de anotação de observabilidade e aplica uma regra de prioridade consistente na aceitação de coordenadores e heartbeats.

Este relatório tem quatro objetivos principais:

- apresentar a arquitetura e as decisões de projeto;
- relacionar as regras teóricas com sua concretização no código;
- descrever a metodologia de validação e interpretar os resultados disponíveis;
- registrar limitações, adaptações e ameaças à validade sem atribuir aos algoritmos garantias que não foram implementadas.

O relatório está organizado da seguinte forma. A Seção 2 apresenta objetivos, requisitos e a fundamentação teórica dos três algoritmos. A Seção 3 descreve a arquitetura, o modelo de mensagens e a API. A Seção 4 detalha a implementação de cada mecanismo. A Seção 5 trata da estratégia de validação, dos testes e dos experimentos. A Seção 6 sintetiza os resultados e discute limitações e extensões.

2 Objetivos, requisitos e fundamentação

2.1 Objetivo geral

O objetivo geral é construir um sistema distribuído pequeno, reproduzível e suficientemente completo para demonstrar comunicação entre processos, ordenação lógica de eventos, exclusão mútua distribuída e eleição de coordenador. A implementação deve permanecer compreensível e inspecionável, privilegiando a correspondência entre teoria, código e testes.

2.2 Requisitos funcionais

Os requisitos funcionais consolidados no projeto são:

- executar três nós a partir do mesmo código, diferenciados por configuração;
- permitir mensagens de aplicação entre quaisquer dois nós;
- manter um relógio lógico independente em cada nó;
- solicitar e liberar uma seção crítica sem um servidor central de autorização;
- eleger o maior identificador ativo como líder;
- detectar a ausência do líder por heartbeat e timeout;
- expor estado e histórico local por API;
- observar entradas e saídas da seção crítica em um serviço externo;
- automatizar cenários de validação e coleta de métricas.

2.3 Requisitos de qualidade

A solução também busca atender requisitos não funcionais:

- **isolamento:** cada nó deve executar em processo e contêiner próprios;
- **reprodutibilidade:** a pilha deve ser iniciada por Docker Compose;
- **observabilidade:** mensagens, transições e falhas devem aparecer nos logs e endpoints;
- **determinismo de prioridade:** empates devem ser resolvidos pelo identificador do nó;
- **testabilidade:** regras puras e transições de estado devem possuir testes unitários;
- **rastreabilidade:** decisões e adaptações devem ser relacionadas às fontes e aos arquivos do projeto.

2.4 Relógios lógicos de Lamport

Lamport define a relação *happened-before*, denotada por $a \rightarrow b$, para expressar que o evento a pode ter influenciado o evento b [1]. A relação inclui a ordem local de eventos em um processo, o envio antes do recebimento da mesma mensagem e o fechamento transitivo dessas relações.

A condição que um relógio lógico deve preservar é:

$$a \rightarrow b \Rightarrow C(a) < C(b). \quad (1)$$

A recíproca não é garantida: $C(a) < C(b)$ não prova que $a \rightarrow b$. Dois eventos concorrentes podem receber timestamps diferentes apenas porque os relógios locais evoluíram de forma distinta.

A concretização inteira empregada pelo projeto utiliza duas operações:

$$C_i \leftarrow C_i + 1 \quad \text{em evento local ou envio;} \quad (2)$$

$$C_i \leftarrow \max(C_i, t_m) + 1 \quad \text{ao receber mensagem com timestamp } t_m. \quad (3)$$

O timestamp inserido na mensagem é o valor do relógio no evento de envio. A granularidade de evento é uma escolha de implementação; por isso, o projeto distingue eventos que avançam o relógio de anotações que apenas detalham o mesmo acontecimento.

2.5 Exclusão mútua de Ricart–Agrawala

No algoritmo de Ricart–Agrawala, um processo que deseja entrar na seção crítica envia um pedido aos demais participantes e aguarda uma resposta de cada um [2]. Pedidos concorrentes são ordenados pelo par:

$$(timestamp_do_pedido, identificador_do_processo). \quad (4)$$

O menor par tem prioridade. Um receptor responde imediatamente quando não disputa o recurso ou quando o pedido remoto tem prioridade. Caso esteja na seção crítica, ou possua um pedido local prioritário, adia a resposta até a liberação.

Para N participantes, uma solicitação bem-sucedida e sem falhas utiliza:

$$(N - 1) \text{ REQUESTs} + (N - 1) \text{ REPLYs} = 2(N - 1) \quad (5)$$

mensagens do protocolo. No cenário com três nós, o valor esperado é quatro. O timeout local implementado no projeto evita espera infinita em testes, mas não torna o algoritmo tolerante a falhas.

2.6 Eleição pelo algoritmo Bully

O algoritmo Bully parte de processos com identificadores totalmente ordenados. Quando um processo detecta a falha do coordenador, envia uma eleição apenas aos processos com identificador maior. Se nenhum deles responder, torna-se coordenador; se algum responder, aguarda o anúncio de um vencedor superior [3].

O projeto usa os tipos `ELECTION`, `ELECTION_OK` e `COORDINATOR` para representar esse fluxo. Dois elementos são adaptações locais:

- `HEARTBEAT`, usado para detecção periódica de falha;
- `election_id`, usado para rejeitar respostas antigas ou duplicadas de uma rodada local.

A implementação assume falha por parada, posterior recuperação e atrasos limitados o suficiente para que os timeouts tenham significado. Ela não oferece tolerância a partições de rede.

2.7 Escopo e exclusões

Permanecem fora do escopo:

- consenso por Paxos ou Raft;
- replicação de dados e persistência durável;
- membership dinâmico;
- autenticação e autorização;
- tolerância a partições e atrasos arbitrários;
- banco de dados, filas externas, Kubernetes, frontend ou dashboard.

Essas exclusões evitam misturar problemas diferentes e preservam o foco nos três algoritmos estudados.

3 Arquitetura do sistema

3.1 Visão geral

A aplicação é composta por quatro serviços definidos em `docker-compose.yml` [14]. Os três nós executam a mesma imagem e o mesmo módulo ASGI; as diferenças são fornecidas pelas variáveis `NODE_ID` e `PEERS`. O quarto serviço, `resource`, executa outro módulo FastAPI e atua somente como observador da seção crítica.

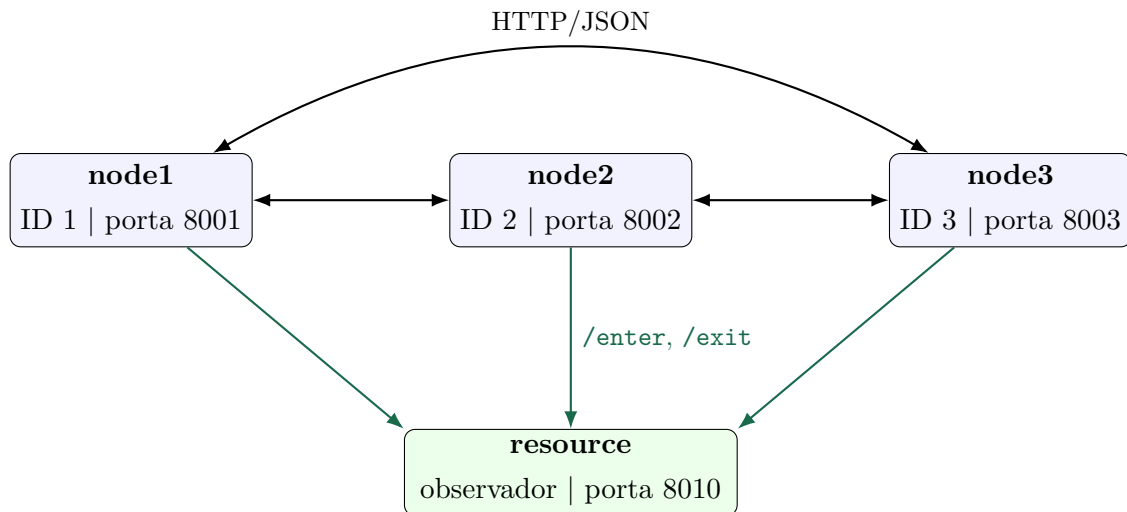


Figura 1: Topologia lógica e serviços expostos no host. As arestas bidirecionais entre os nós formam uma malha completa e transportam mensagens de aplicação, mutex, eleição e heartbeat.

As arestas entre os nós representam mensagens de aplicação, mutex, eleição e heartbeat. As setas para **resource** são notificações de entrada e saída da seção crítica; não existe seta de volta, porque o observador não concede permissão.

3.2 Componentes e responsabilidades

Componente	Responsabilidade e observações
<code>src/node/main.py</code>	Expõe a API, recebe comandos, despacha mensagens e executa os fluxos da seção crítica, eleição e heartbeat.
<code>src/node/state.py</code>	Mantém o estado concorrente do nó e protege relógio, mutex, líder e histórico por <code>asyncio.Lock</code> .
<code>src/node/clock.py</code>	Implementa incremento local e atualização do relógio no recebimento.
<code>src/node/mutex.py</code>	Define estados, prioridade e regra de adiamento de respostas.
<code>src/node/election.py</code>	Valida timeouts, seleciona peers superiores e decide aceitação de líder.
<code>src/node/models.py</code>	Define mensagens, comandos e registros validados por Pydantic.
<code>src/node/transport.py</code>	Serializa o envelope e realiza chamadas HTTP com HTTPX.
<code>src/resource/main.py</code>	Registra ENTER, EXIT e sobreposição, sem autorizar acesso.

Tabela 1: Divisão de responsabilidades no código.

3.3 Estrutura interna de um nó

Cada nó mantém quatro grupos de estado: configuração de peers, relógio e contadores, exclusão mútua e eleição. A Figura 2 resume o fluxo entre as camadas.

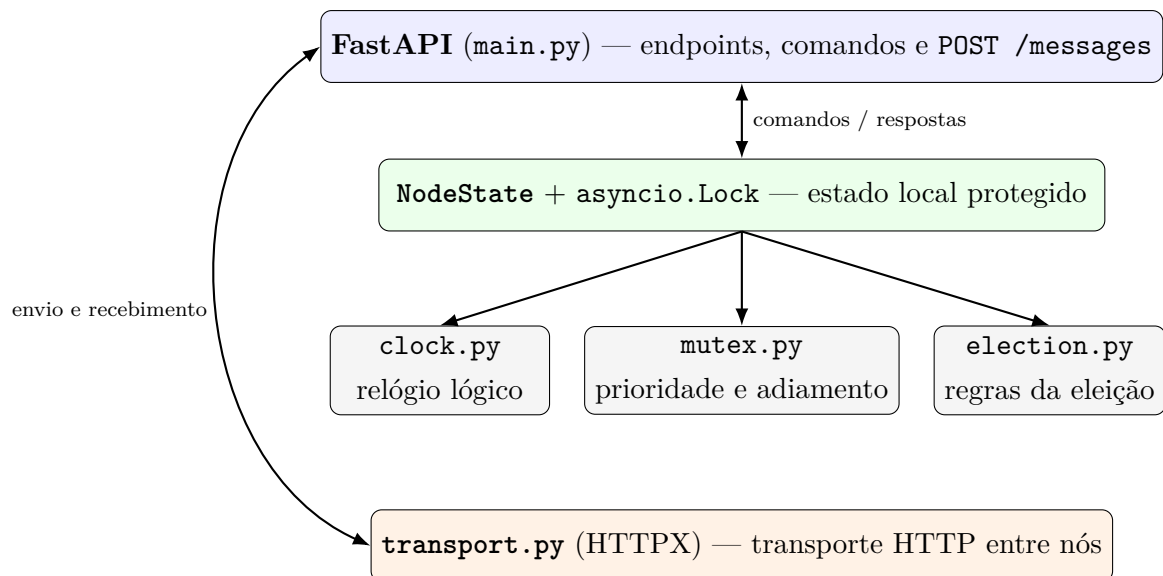


Figura 2: Organização interna de cada nó em camadas. A API delega o transporte HTTP a `transport.py` e concentra o estado em `NodeState`, que aplica as funções puras de `clock.py`, `mutex.py` e `election.py`.

O lock do estado não substitui os algoritmos distribuídos. Ele apenas evita corridas entre corrotinas do mesmo processo ao modificar estruturas locais, como o conjunto de respostas recebidas ou o valor do relógio. A exclusão entre processos continua dependendo das mensagens do protocolo.

3.4 Envelope de mensagens

Todos os protocolos reutilizam `AppMessage`. A Tabela 2 apresenta os campos.

Campo	Tipo	Função
<code>type</code>	texto	Identifica mensagem de aplicação, mutex, eleição ou heartbeat.
<code>sender_id</code>	inteiro	Identifica o processo que enviou a mensagem.
<code>message_id</code>	texto	UUID da mensagem, usado para correlação e observabilidade.
<code>logical_time</code>	inteiro não negativo	Timestamp do evento lógico de envio.
<code>payload</code>	objeto JSON	Dados específicos, como <code>request_id</code> , <code>request_timestamp</code> , <code>election_id</code> ou <code>leader_id</code> .

Tabela 2: Envelope comum às mensagens entre nós.

O modelo é validado por Pydantic [10]. Essa validação impede, por exemplo, timestamps negativos e durações da seção crítica fora do intervalo aceito.

3.5 API e observabilidade

Tabela 3: Endpoints principais dos nós.

Endpoint	Método	Finalidade
<code>/health</code>	GET	Verifica disponibilidade e retorna o ID do nó.
<code>/state</code>	GET	Expõe snapshot do relógio, contadores, mutex e eleição.
<code>/events</code>	GET	Retorna o log local de eventos e anotações.
<code>/messages</code>	POST	Recebe o envelope comum e despacha pelo tipo.
<code>/commands/local-event</code>	POST	Gera um evento local de Lamport.
<code>/commands/send-message</code>	POST	Envia uma mensagem de aplicação a um peer.
<code>/commands/request-critical-section</code>	POST	Executa uma tentativa completa do mutex.
<code>/commands/start-election</code>	POST	Agenda uma eleição manual.

Os registros de `/events` possuem `event_kind`. O valor `lamport_event` identifica os eventos centrais usados na demonstração causal; `annotation` identifica detalhes e transições auxiliares do protocolo. A classificação é informativa e não deve ser usada

isoladamente para inferir se uma função chamou `tick()`; ela evita, sobretudo, tratar cada linha do log como um evento teórico independente.

3.6 Configuração temporal

Variável	Valor	Papel
HEARTBEAT_INTERVAL_MS	700 ms	Intervalo entre heartbeats do líder.
LEADER_TIMEOUT_MS	2500 ms	Limite sem heartbeat antes de iniciar eleição.
ELECTION_RESPONSE_TIMEOUT_MS	700 ms	Espera por resposta de processo superior.
COORDINATOR_TIMEOUT_MS	2500 ms	Espera pelo anúncio após receber resposta.
STARTUP_ELECTION_DELAY_MS	500 ms	Atraso inicial antes da eleição de startup.

Tabela 4: Timeouts configurados no Docker Compose.

Os intervalos são medidos com `time.monotonic()`, apropriado para diferenças de tempo porque não depende de ajustes do relógio de parede [6].

4 Desenvolvimento e implementação

4.1 Base de execução

O projeto utiliza Python 3.12 e empacotamento definido em `pyproject.toml`. FastAPI fornece os endpoints e o ciclo de vida ASGI; Uvicorn executa as aplicações; HTTPX realiza as chamadas entre processos; Pydantic valida os contratos; Pytest e pytest-asyncio compõem a suíte de testes [7, 8, 9, 10, 11, 12].

O `Dockerfile` [13] produz uma imagem comum. No Compose, `node1` realiza o build e os demais serviços reutilizam a imagem. Cada nó recebe uma lista de peers com os endereços internos `http://nodeX:8000`. As portas 8001, 8002, 8003 e 8010 são apenas mapeamentos para acesso pelo host.

4.2 Estado local e sincronização entre corrotinas

A classe `NodeState` concentra o estado mutável e usa um único `asyncio.Lock`. A documentação de Python define `Lock` como uma primitiva de exclusão mútua entre tarefas assíncronas e `Event` como uma sinalização para tarefas em espera [4]. No projeto, o lock protege:

- valor do relógio e contadores de mensagens;

- estado, timestamp e respostas do mutex;
- líder conhecido, rodada e respostas de eleição;
- histórico circular de até 500 registros.

Um `asyncio.Event` é criado em cada pedido de mutex. Ele é sinalizado quando o conjunto de respostas atinge o número de peers. O endpoint usa `asyncio.wait_for()` para limitar a espera, sem implementar polling ativo [5].

4.3 Relógio de Lamport e log de eventos

4.3.1 Operações do relógio

O módulo `clock.py` mantém um contador inteiro iniciado em zero. As operações são pequenas e determinísticas:

Listing 1: Operações essenciais do relógio lógico.

```
def tick(self) -> int:
    self.value += 1
    return self.value

def update(self, received_time: int) -> int:
    self.value = max(self.value, received_time) + 1
    return self.value
```

Um envio executa `tick()`, cria a mensagem com o valor obtido e registra **SEND** com o mesmo timestamp. Um recebimento executa `update()` antes de registrar **RECEIVE**. Assim, o timestamp local do recebimento é sempre maior que o valor transportado.

4.3.2 Eventos e anotações

O histórico não é uma cópia literal da definição matemática de evento. Para separar a linha causal principal dos detalhes do protocolo, o modelo distingue:

- **lamport_event**: evento central usado na linha causal, como **LOCAL**, **SEND**, **RECEIVE**, **MUTEX_WANTED**, **ENTER_CS**, **EXIT_CS**, **ELECTION_STARTED** e **LEADER_TIMEOUT**;
- **annotation**: detalhe ou transição auxiliar, como uma resposta adiada, uma mudança de líder, a rejeição de coordenador ou uma falha de envio.

Por exemplo, o recebimento de um pedido de mutex gera um **RECEIVE** e uma anotação **MUTEX_REQUEST_RECEIVED** com o mesmo timestamp. Isso representa um único evento de recebimento e duas visões de observabilidade, não dois eventos sucessivos do processo. A classificação é semântica, não um mecanismo de controle do relógio: a transição

MUTEX_HELD, por exemplo, executa `tick()` e ainda assim é rotulada como `annotation`. Portanto, reconstruções precisas devem considerar a ação e o timestamp, e não apenas `event_kind`.

4.3.3 Exemplo causal

Considere o encadeamento usado no smoke test:

1. `node1` registra um evento local;
2. `node1` envia uma mensagem a `node2`;
3. `node2` recebe a mensagem e depois envia outra a `node3`;
4. `node3` recebe a segunda mensagem.

O script verifica as desigualdades:

$$C(S_{1 \rightarrow 2}) < C(R_{1 \rightarrow 2}),$$

$$C(R_{1 \rightarrow 2}) < C(S_{2 \rightarrow 3}),$$

$$C(S_{2 \rightarrow 3}) < C(R_{2 \rightarrow 3}).$$

Essas relações demonstram a condição de Lamport no cenário observado. Elas não provam que qualquer par de timestamps ordenados possua relação causal.

4.4 Exclusão mútua distribuída

4.4.1 Modelagem de estados

A implementação representa o estado local por três valores:

Estado	Significado	Comportamento ao receber pedido
RELEASED	Não deseja e não ocupa a seção crítica.	Responde imediatamente.
WANTED	Possui pedido local aguardando respostas.	Compara prioridades e pode adiar.
HELD	Está dentro da seção crítica.	Sempre adia a resposta.

Tabela 5: Estados locais usados para Ricart–Agrawala.

Esses nomes são a modelagem adotada no projeto; a referência acadêmica é usada para as regras do algoritmo, não para afirmar que a nomenclatura seja textual no artigo original.

4.4.2 Criação e disseminação do pedido

Ao iniciar uma tentativa, o nó:

1. exige estado `RELEASED`;
2. incrementa o relógio e fixa esse valor em `request_timestamp`;
3. gera um `request_id` único;
4. muda para `WANTED`;
5. limpa respostas antigas;
6. envia `MUTEX_REQUEST` a todos os peers.

O `request_timestamp` é igual para todos os pedidos da mesma tentativa. Já o `logical_time` do envelope muda a cada envio, pois cada transmissão é um evento distinto. Essa separação é necessária: a prioridade pertence à tentativa, e não ao instante em que cada peer foi contatado.

4.4.3 Regra de prioridade

A função pura de prioridade usa comparação lexicográfica:

$$local_priority \Leftrightarrow (t_{local}, id_{local}) < (t_{remoto}, id_{remoto}). \quad (6)$$

Se o estado local for `WANTED` e o pedido local tiver prioridade, a resposta remota é armazenada em `deferred_replies`. Caso contrário, `MUTEX_REPLY` é enviada imediatamente. Conjuntos impedem que respostas duplicadas sejam contadas mais de uma vez, e o `request_id` impede que uma resposta antiga libere uma tentativa nova.

4.4.4 Entrada, saída e abort

A passagem para `HELD` só ocorre depois de receber exatamente uma resposta de cada peer. A entrada e a saída da seção crítica são eventos locais separados. Após a saída, o estado volta a `RELEASED` e todas as respostas adiadas são enviadas.

Se a espera ultrapassa o timeout, a tentativa é encerrada por `MUTEX_ABORTED`. Essa ação incrementa o relógio, limpa o estado local e devolve as respostas adiadas. O procedimento melhora a controlabilidade dos testes, porém não garante que todos os processos tenham visão consistente do abort nem resolve falha de participante.

4.4.5 Observador externo

Antes e depois do período simulado na seção crítica, o nó envia notificações a `resource`. O observador mantém um mapa de solicitações ativas. Uma nova entrada enquanto o mapa não está vazio incrementa `violations`. Esse mecanismo fornece evidência

de ausência de sobreposição nos cenários executados, mas não faz parte da autorização e não é uma prova formal para todas as execuções possíveis.

4.5 Eleição de líder

4.5.1 Mensagens e fluxo

A Tabela 6 relaciona as mensagens com sua função.

Tipo	Função	Origem
ELECTION	Pergunta a processos de ID maior se algum está ativo.	Regra central do Bully.
ELECTION_OK	Confirma que um processo superior está ativo.	Nome local para resposta superior.
COORDINATOR	Anuncia o vencedor da eleição.	Regra central do Bully.
HEARTBEAT	Renova a presença do líder conhecido.	Adaptação do projeto.

Tabela 6: Mensagens do subsistema de eleição.

Ao iniciar uma rodada, o nó gera um UUID em `election_id`, incrementa o relógio e seleciona apenas peers de ID maior. Se a lista estiver vazia, torna-se líder e anuncia o resultado. Se nenhum superior responder dentro de 700 ms, o iniciador também se declara líder. Quando recebe ao menos uma resposta, espera até 2500 ms por `COORDINATOR`; na ausência do anúncio, registra `ELECTION_ROUND_TIMED_OUT` e inicia uma nova rodada.

4.5.2 Detecção de falha e recuperação

O líder envia heartbeat a cada 700 ms. Seguidores usam `time.monotonic()` para medir o tempo desde a última mensagem aceita. Após 2500 ms, removem o líder, registram `LEADER_TIMEOUT` e agendam uma eleição. No startup, cada nó também agenda uma eleição após 500 ms; isso permite que um processo superior recuperado volte a disputar a liderança.

4.5.3 Aceitação de líder

A aceitação de um coordenador não depende apenas da existência de uma eleição em andamento; ela respeita a prioridade dos identificadores. A regra implementada é:

$$\begin{aligned} \text{accept}(\text{announced}) \Leftrightarrow & \text{announced} \geq \text{node_id} \\ & \wedge (\text{current} = \emptyset \vee \text{announced} \geq \text{current}). \end{aligned} \quad (7)$$

A primeira condição impede que um processo ativo aceite como líder alguém inferior a si próprio. A segunda impede rebaixar um líder já conhecido. O heartbeat exige ainda que `sender_id == leader_id`.

Se um coordenador ou heartbeat coerente, porém inferior ao nó local, é rejeitado, a camada de controle agenda uma eleição. O agendamento usa uma task única protegida por lock, evitando a criação simultânea de várias tarefas locais. Mensagens malformadas, como `leader_id` diferente do remetente, são rejeitadas sem serem tratadas como evidência de uma liderança inferior válida.

4.5.4 Limites do identificador de eleição

O `election_id` é associado à rodada local do iniciador e permite ignorar `ELECTION_OK` duplicado, atrasado ou referente a outra tentativa. Ele não é um número de época global nem identifica todas as eleições concorrentes do sistema. A convergência depende da regra de prioridade, do anúncio do coordenador e dos timeouts.

4.6 Decisões de implementação e limitações

As decisões centrais foram manter o estado em memória, evitar qualquer coordenador para o mutex, reutilizar um envelope comum e realizar a falha do líder pela parada real do contêiner. Como consequência:

- reiniciar um nó perde seu histórico e estado local;
- falhas durante uma rodada de mutex podem interromper o progresso;
- os timeouts foram calibrados para a execução local;
- uma partição pode criar visões incompatíveis de liderança;
- o serviço `resource` pode detectar sobreposição observada, mas não impedir uma violação;
- a criação de um novo `httpx.AsyncClient` por chamada é simples, porém não aproveita pooling de longo prazo sugerido pela documentação do HTTPX [9].

5 Validação, testes e experimentos

5.1 Estratégia de validação

A validação foi organizada em três níveis complementares:

1. **testes unitários:** verificam funções puras e transições de estado sem depender da rede;

2. **smoke tests distribuídos**: exercitam os serviços reais em contêineres, usando apenas APIs públicas e Docker Compose;
3. **experimentos repetidos**: executam cenários cinco vezes, armazenam dados brutos e calculam estatísticas descritivas.

Essa separação permite localizar erros. Um teste de prioridade pode falhar sem iniciar contêineres; um problema de transporte aparece nos smoke tests; variações temporais são observadas nos experimentos.

5.2 Testes unitários

A suíte de testes unitários, executada na virtualenv do projeto, conclui com **62 testes aprovados**. Os testes cobrem:

- incremento e atualização do relógio;
- validação de modelos e configuração;
- prioridade e adiamento em Ricart–Agrawala;
- transições RELEASED, WANTED e HELD;
- rejeição de respostas duplicadas e antigas;
- abort do mutex e liberação de respostas adiadas;
- seleção de peers superiores no Bully;
- aceitação e rejeição de coordenador e heartbeat;
- preservação de uma eleição ativa após mensagem inferior;
- classificação entre `lamport_event` e `annotation`;
- monotonicidade dos timestamps no histórico local.

Uma matriz parametrizada verifica casos essenciais da regra de coordenador, incluindo nó local superior ao anúncio, líder atual superior ao anúncio e aceitação de um coordenador maior durante eleição ativa. Testes adicionais confirmam que mensagens de eleição não alteram o estado do mutex.

5.3 Smoke tests

Os scripts exercitam a pilha como uma aplicação distribuída:

Script	Cenário verificado
<code>scripts/smoke_http.py</code>	Envio HTTP/JSON de <code>node1</code> para <code>node2</code> , contadores e evento de recebimento.
<code>scripts/smoke_lamport.py</code>	Evento local e cadeia causal <code>node1 -> node2 -> node3</code> .
<code>scripts/smoke_mutex.py</code>	Pedido individual, três pedidos concorrentes, quatro mensagens no caso individual e ausência de sobreposição observada.
<code>scripts/smoke_election.py</code>	Líder inicial 3, parada real do <code>node3</code> , eleição do 2, recuperação do 3 e nova convergência.
<code>scripts/demo.py</code>	Encadeamento dos quatro smoke tests em uma única execução.

Tabela 7: Cenários cobertos pelos smoke tests.

Os scripts esperam explicitamente pela saúde dos serviços e fazem polling apenas para observar estados assíncronos. Eles não modificam diretamente variáveis internas dos nós.

5.4 Metodologia experimental

O script `run_experiments.py` gera três grupos de experimentos:

- cinco repetições da cadeia causal de Lamport;
- cinco pedidos individuais e cinco rodadas com três pedidos concorrentes de mutex;
- cinco ciclos completos de eleição, parada do líder e recuperação.

Os arquivos versionados são:

- `docs/results/raw-results.json`, com eventos e estados detalhados;
- `docs/results/experiment-summary.csv`, com estatísticas;
- `docs/results/README.md`, com ambiente e comandos.

Os experimentos versionados foram executados em **4 de julho de 2026 (UTC)**. As médias, medianas, mínimos e máximos foram calculados pelo próprio script a partir da pilha em execução.

5.5 Resultados experimentais versionados

Experimento	Métrica	n	Mín.	Máx.	Média	Mediana
Lamport	violações	5	0	0	0	0
Mutex individual	mensagens	5	4	4	4	4
Mutex individual	espera (ms)	5	51,497	107,261	71,692	69,893
Mutex individual	duração observada (ms)	5	162	166	164,6	165
Mutex concorrente	espera (ms)	15	92,391	656,909	368,799	365,407
Mutex concorrente	duração observada (ms)	15	213	218	215,267	215
Mutex concorrente	respostas adiadas	15	0	2	1	1
Eleição	detecção/convergência	5	3890	4906	4293,6	4391
	após parada (ms)					
Eleição	recuperação do líder 3 (ms)	5	578	844	662,6	625
Eleição	ciclo total (ms)	5	4531	5624	4999,8	5000

Tabela 8: Resumo da execução experimental versionada.

5.6 Análise dos resultados

5.6.1 Lamport

As cinco repetições não apresentaram violação das relações causais verificadas. O resultado confirma que, nos cenários executados, o timestamp de envio foi menor que o de recebimento e que o envio posterior de **node2** ocorreu depois do recebimento anterior. A métrica não verifica todos os pares de eventos nem constitui prova completa do sistema.

5.6.2 Exclusão mútua

Todos os pedidos individuais usaram exatamente quatro mensagens, em acordo com a Equação 5 para $N = 3$. A espera média foi de aproximadamente 71,7 ms, enquanto o período observado pelo recurso ficou próximo dos 150 ms solicitados, acrescido do custo de requisições e agendamento.

Nos pedidos concorrentes, não houve sobreposição em 15 entradas. A espera média aumentou para 368,8 ms porque os processos foram serializados e respostas precisaram ser adiadas. A média de uma resposta adiada por solicitação é compatível com a existência de disputa, mas depende da ordem concreta de chegada das mensagens.

5.6.3 Eleição

Após parar **node3**, a convergência média para **node2** foi de 4,294 s. Esse tempo é superior ao `LEADER_TIMEOUT_MS` isolado porque inclui polling, detecção, mensagens de eleição e estabilização observada por todos os nós restantes. A recuperação de **node3** foi mais rápida, com média de 662,6 ms, porque a eleição de startup é agendada 500 ms após o reinício e o nó recuperado não possui peer superior.

Em todas as repetições, os nós ativos convergiram para o mesmo líder ao final. O resultado não cobre partições, perda arbitrária de mensagens ou atrasos superiores aos timeouts.

5.7 Reprodutibilidade

A sequência recomendada é:

Listing 2: Comandos de validação e reprodução.

```
python -m venv .venv
.venv/Scripts/python -m pip install -e ".[test]"
.venv/Scripts/python -m pytest -q

docker compose up --build -d
python scripts/smoke_http.py
python scripts/smoke_lamport.py
python scripts/smoke_mutex.py
python scripts/smoke_election.py
python scripts/run_experiments.py
docker compose down
```

Em Linux, o executável da virtualenv normalmente fica em `.venv/bin/python`. O script de experimentos executa `docker compose down` no bloco de limpeza para reduzir o risco de deixar a pilha ativa após falha.

5.8 Ameaças à validade

As principais ameaças à validade são:

- apenas três processos e execução na mesma máquina;
- amostra pequena, com cinco repetições por grupo;
- medição por relógio de parede em parte das métricas de duração;
- contêineres e rede Docker com baixa latência em relação a uma rede real;
- observação de segurança por um serviço externo, e não verificação formal;
- ausência de injeção sistemática de perda, duplicação, reordenação e partição.

A documentação auxiliar em `docs/algorithm-traceability.md`, `docs/decision-log.md` e `docs/references.md` complementa esta análise ao relacionar regras, funções, testes e decisões.

6 Conclusão

O trabalho produziu uma implementação integrada de conceitos clássicos de sistemas distribuídos em uma pilha pequena e reproduzível. Os três nós executam processos independentes, trocam mensagens HTTP/JSON e mantêm estado exclusivamente local. Sobre essa base foram construídos o relógio lógico de Lamport, a exclusão mútua de Ricart–Agrawala e uma adaptação do algoritmo Bully.

Do ponto de vista de engenharia, os principais resultados são:

- um envelope comum para mensagens de aplicação, mutex e eleição;
- atualização consistente do relógio em eventos locais, envios e recebimentos;
- separação explícita entre eventos Lamport e anotações de observabilidade;
- prioridade determinística de pedidos por timestamp e ID;
- observação externa da seção crítica sem centralizar a autorização;
- eleição, detecção de parada e recuperação do maior identificador;
- rejeição de coordenadores e heartbeats inferiores ao próprio nó ou ao líder conhecido;
- testes unitários, smoke tests e experimentos com artefatos versionados.

O relatório também esclarece duas interpretações incorretas. Primeiro, o histórico de `/events` não é uma sequência um-para-um de eventos teóricos: anotações podem compartilhar o timestamp do acontecimento que descrevem. Segundo, os timeouts empregados no mutex e na eleição são hipóteses operacionais da demonstração, não garantias de tolerância a falhas em um sistema assíncrono arbitrário.

Os resultados experimentais são coerentes com os comportamentos esperados: zero violações causais no cenário monitorado, quatro mensagens para um pedido individual de mutex com três nós, nenhuma sobreposição observada na seção crítica e convergência do líder após parada e recuperação.

6.1 Limitações e possíveis extensões

O sistema continua limitado a participantes fixos, estado volátil, rede local e falha por parada. Não há solução para partições, consenso, replicação ou persistência. Como extensões futuras, seria possível:

- introduzir testes de caos para atraso, perda e reordenação de mensagens;
- reutilizar um `httpx.AsyncClient` de longa duração para pooling de conexões;
- ampliar os experimentos e reportar intervalos de confiança;
- adicionar identificadores de época mais fortes ao subsistema de eleição;

- comparar o Bully com um protocolo de consenso ou uma eleição baseada em log, mantendo clara a mudança de escopo;
- adicionar persistência apenas para fins de recuperação, sem confundi-la com o estado compartilhado proibido no núcleo da atividade.

Dentro do escopo proposto, a implementação oferece uma demonstração clara da relação entre teoria, mensagens reais, estado local e comportamento observável. O conjunto formado por código, testes, scripts, documentação e resultados permite reproduzir os cenários e discutir tanto as propriedades alcançadas quanto as garantias que permanecem fora do modelo.

7 Fontes de código e declaração de autoria

Todo o código deste trabalho foi escrito pelos autores. Os três algoritmos — relógio lógico de Lamport, exclusão mútua de Ricart–Agrawala e eleição pelo algoritmo Bully — foram implementados a partir dos artigos originais citados na Seção 2 [1, 2, 3], sem reutilizar ou adaptar implementações prontas de terceiros. Nenhum trecho foi copiado de repositórios externos, fóruns ou soluções publicadas dos algoritmos.

A configuração e o uso das bibliotecas seguem os exemplos das documentações oficiais referenciadas neste relatório, adaptados às necessidades do projeto:

Origem	Fonte	Adaptação feita pelos autores
Aplicação ASGI e lifespan	FastAPI [7]	Endpoints próprios (/messages , /commands/*); o lifespan inicia e encerra as tarefas de heartbeat e de eleição.
Cliente HTTP assíncrono	HTTPX [9]	Envio do envelope AppMessage entre nós e notificação ao observador.
Modelos e validação	Pydantic [10]	Definição de AppMessage , dos comandos e do registro de eventos, com restrições (timestamp não negativo, duração da seção crítica).
Imagem e orquestração	Dockerfile [13], Compose [14]	Imagem única reaproveitada pelos quatro serviços; NODE_ID , PEERS , timeouts e healthcheck definidos pelos autores.
Servidor ASGI	Uvicorn [8]	Execução dos módulos com -host 0.0.0.0 -port 8000 .

Tabela 9: Fontes de código de terceiros e alterações realizadas.

Em resumo, o material de terceiros restringe-se ao uso de bibliotecas por meio de suas APIs e exemplos oficiais; a lógica dos algoritmos, o modelo de estado e os testes são de autoria própria.

8 Uso de ferramentas de inteligência artificial

Durante o desenvolvimento deste trabalho, os autores utilizaram ferramentas de inteligência artificial generativa como apoio à elaboração e revisão do relatório, ao desenvolvimento e à revisão do código e dos testes, e à produção da documentação auxiliar. As ferramentas foram empregadas como assistentes de escrita, verificação e revisão, e não como substitutas do raciocínio ou das decisões de projeto.

Todo o conteúdo produzido ou sugerido com esse apoio foi revisado, validado e, quando necessário, corrigido pelos autores, que assumem integral responsabilidade pelo texto, pelo código, pelos resultados e pelas conclusões apresentados. Nenhuma implementação externa dos algoritmos foi incorporada: a solução foi construída a partir das fontes acadêmicas e da documentação oficial referenciadas neste relatório.

9 Referências

Referências

- [1] LAMPORT, Leslie. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM, v. 21, n. 7, p. 558–565, 1978. DOI: <https://doi.org/10.1145/359545.359563>. Texto consultado: <https://lamport.azurewebsites.net/pubs/time-clocks.pdf>.
- [2] RICART, Glenn; AGRAWALA, Ashok K. *An Optimal Algorithm for Mutual Exclusion in Computer Networks*. Communications of the ACM, v. 24, n. 1, p. 9–17, 1981. DOI: <https://doi.org/10.1145/358527.358537>.
- [3] GARCIA-MOLINA, Hector. *Elections in a Distributed Computing System*. IEEE Transactions on Computers, v. C-31, n. 1, p. 48–59, 1982. DOI: <https://doi.org/10.1109/TC.1982.1675885>.
- [4] PYTHON SOFTWARE FOUNDATION. *Synchronization Primitives*. Disponível em: <https://docs.python.org/3.12/library/asyncio-sync.html>. Acesso em: 5 jul. 2026.
- [5] PYTHON SOFTWARE FOUNDATION. *asyncio.wait_for*. Disponível em: https://docs.python.org/3.12/library/asyncio-task.html#asyncio.wait_for. Acesso em: 5 jul. 2026.
- [6] PYTHON SOFTWARE FOUNDATION. *time.monotonic*. Disponível em: <https://docs.python.org/3.12/library/time.html#time.monotonic>. Acesso em: 5 jul. 2026.
- [7] FASTAPI. *First Steps e Lifespan Events*. Disponível em: <https://fastapi.tiangolo.com/tutorial/first-steps/> e <https://fastapi.tiangolo.com/advanced/events/>. Acesso em: 5 jul. 2026.
- [8] UVICORN. *Settings*. Disponível em: <https://www.uvicorn.org/settings/>. Acesso em: 5 jul. 2026.
- [9] HTTPX. *Async Support*. Disponível em: <https://www.python-httpx.org/async/>. Acesso em: 5 jul. 2026.
- [10] PYDANTIC. *Models*. Disponível em: <https://docs.pydantic.dev/latest/concepts/models/>. Acesso em: 5 jul. 2026.
- [11] PYTEST. *Documentation*. Disponível em: <https://docs.pytest.org/en/stable/>. Acesso em: 5 jul. 2026.

- [12] PYTEST-ASYNCIO. *Configuration*. Disponível em: <https://pytest-asyncio.readthedocs.io/en/stable/reference/configuration.html>. Acesso em: 5 jul. 2026.
- [13] DOCKER. *Dockerfile Reference*. Disponível em: <https://docs.docker.com/reference/dockerfile/>. Acesso em: 5 jul. 2026.
- [14] DOCKER. *Compose File Reference: Services*. Disponível em: <https://docs.docker.com/reference/compose-file/services/>. Acesso em: 5 jul. 2026.